

一、概述

1、Java 程序的开发过程:

➤ 编辑 (*^{源文件}.java) → 编译 (*^{字节码文件}.class) → 解释执行

2、JDK: java 开发工具包;

开发工具
Java 基础类库
JVM } JDK

3、JRE: java 运行时环境;

4、JVM: Java 虚拟机, 使得 Java 语言实现平台无关性。

二、Java 编程基础

1、基本数据类型、变量、常量

1) ^{1字节存储}byte ^{2字节}char ^{4字节}short ^{8字节}int ^{4字节}long ^{4字节}float ^{8字节}double ^{1字节}boolean (true/false)

2) 每种类型有特定取值范围

3) 常量数据类型: 按其值的大小和书写形式确定类型 100 200

4) 变量定义、命名规则: 数据类型 变量名;

➤ 由字母、数字、下划线、美元符号组成, 且第一个字符不能是数字;

➤ 关键字不能作为标识符: abstract, private, do, boolean, double, import, public, byte, this, static, final, void

5) 类型转换

➤ 整数类型之间: 小范围到大范围的自动转换; 大范围到小范围需要强制转换; 目标类型 变量 = (目标类型) 值

➤ 小数给整数类型变量赋值: 需要强制转换;

6) final 声明常量: final int a = 3; 常量只能被赋值一次。后续不能修改

2、运算符 修饰变量/类/成员方法

1) 算术运算符 + - * / % ++ -- = += /= %=

➤ 两个整数类型的数据相除, 结果取整 (舍去小数部分);

2) 关系运算符 > >= < <= == !=

➤ 关系表达式的值: 布尔类型, 取值为 true 或 false;

3) 逻辑运算符 && || !

➤ 参与逻辑运算的必须是布尔型数据或表达式;

➤ 逻辑表达式的值: 布尔类型, 取值为 true 或 false;

4) 条件运算符 ? : 嵌套后从右往左看

5) 字符串连接运算符: 当+运算的两个数据对象有一个是字符串类型, 则+是字符串连接运算符。

3、控制语句

1) if~else switch

2) while do~while for

4、方法

1) 方法的定义 静态方法

[访问权限] [static] [final] 数据类型 方法名 (参数列表) { 方法体 }

2) 方法的调用: 方法名(实参列表)

3) 方法的重载: 在一个类中可以定义名字相同但参数个数不同或者参数类型不同的多个方法; 调用时根据实参的个数和类型选择一个确定的方法。

5、数组 引用类型 (类类型的对象)

1) 一维数组:

➤ 定义和创建方式: 数据类型 ^{无长度} 数组名 = new 数据类型[长度];

➤ 元素访问方式: 数组名[下标], 访问下标从 0 开始, 访问不能越界;

➤ 数组的长度: 通过 “数组名.length” 的方式获得数组的长度。

2) 二维数组:

➤ 定义和创建方式:

方法1: int[][] a = new int[10][2];

方法2: int[][] b = new int[2][]; b[0] = new int[3]; b[1] = new int[4];

➤ 访问方式：数组名[行下标][列下标]；行、列下标都从 0 开始，访问不能越界。

3) foreach 语句：遍历数组或集合

书写格式：for(数据类型 变量名：待访问数组名)

{ 循环体 //根据定义的变量迭代访问每个元素 }

6、字符串：掌握 String 类的使用方法

1) String 对象的声明、初始化、创建

String s1 = "abc"; String s2 = new String("abc");

2) String 类中的常用方法

s1.length() s1.substring(1) s1.substring(0,2) s1.charAt(1)

7、数据输入和输出

1) 数据输出：System.out.println(.....); System.out.printf(.....);

2) 数据输入：Scanner sc = new Scanner(System.in); 创建

sc.nextInt(); sc.nextFloat();

三、面向对象基础

1、class：定义类

最终类(无子类)

非抽象类：[访问权限] [final] class 类名 {定义成员变量；定义成员方法；定义构造方法 }

1) 类的两种访问权限：public，说明该类可以被其他所有类访问；缺省的，说明该类只能被同一个包内的其他类访问；

2) final 修饰类：该类不能被继承。

2、成员变量：说明类具有的数据属性

1) 声明方式：[访问权限] [static] [final] 数据类型 变量名；

2) 成员变量访问权限：public protected private 缺省 未指定访问权限

3) static 修饰的成员变量称为类变量，非 static 成员变量称为实例变量；类变量可以通过类名直接访问，实例变量只有在创建对象后通过对象名访问；创建类对象才能调用

4) 成员变量可以是简单数据类型，也可以是类类型。

3、成员方法：说明类具有的行为

1) 非抽象方法的定义：[访问权限] [static] [final] 数据类型 方法名 (参数列表) { 方法体 }

2) 成员方法访问权限：public protected private 缺省

3) static 修饰的成员方法称为类方法，类方法可以通过类名直接访问；

4) final 修饰的成员方法不能被子类重写；

5) 实例成员方法中可以访问实例成员变量；当成员变量和局部变量同名时，默认使用局部变量；

6) 可以使用 this 关键字访问成员变量。

4、构造方法：

1) 定义格式：[访问权限] 方法名 (参数列表) { 构造方法体 }

2) 特点：

➤ 方法名与类名相同

➤ 没有方法数据类型说明符，方法体中没有 return 语句

➤ 可以重载构造方法；重载构造方法的情况下，可以在一个构造方法内的第一行使用 this (参数) 的形式调用该类的其他构造方法；

➤ 不能在成员方法中调用构造方法。

5、new：创建对象

1) 定义并创建对象：类名 对象名 = new 构造方法 (实参列表)；

2) 创建对象时 JVM 为对象的所有实例成员变量分配存储空间，然后通过 new 调用类的构造方法；

3) 创建对象后，通过对象名访问其成员变量和成员方法。

6、import 和 package:

- 1) 包: java 文件的组中方式
- 2) package 语句
 - 功能: 将一个文件中的类加入到一个包中;
 - 语句格式: package 包名; package 包名 1.包名 2.……
- 3) import 语句
 - 功能: 定义一个类时, 如果用到了其他包中的类资源, 则需要通过 import 语句将其引入到当前文件中来;
 - 语句格式: import 引用类所在包名.类名; import 引用类所在包名.*;

四、面向对象进阶

1、继承

- 1) 继承的含义: 在已有类的基础上, 定义新的类, 新类拥有已有类的全部内容, 并且可以添加新的内容; 新定义的类称为子类, 已有类称为父类
- 2) 子类的定义: [访问权限] [final] class 子类名 extends 父类名 { 子类类体 }
- 3) 几点说明:
 - Java 支持单继承, 即一个子类只能有一个直接父类, 定义子类时使用 extends 指出其父类名, extends 后面只能跟一个父类名
 - 对于父类中已经定义的内容, 子类中无需重复定义
 - 在子类中可以直接使用父类中定义的非 private 类型的成员变量和方法
 - 在子类的构造方法中, 默认调用父类的无参构造方法 super(); 也可以书写语句调用父类带参数的构造方法: super (实参列表); 调用父类构造方法的语句必须作为子类构造方法中的第一条语句
- 4) 重写/覆盖: 对于父类中已经定义的成员变量和方法, 可以在子类中重新定义
 - 子类中访问的成员变量和方法默认是在子类中重新定义的变量或方法
 - 成员方法的重写: 在子类中定义和父类中名字相同、方法类型相同、参数个数和类型相同、只有方法体不同的现象;
 - 可以通过 super 关键字访问父类中的被子类重写的成员变量和方法, 访问格式: super.父类成员变量名 或 super.父类成员方法名(实参列表)

2、多态:

- 1) 多态的含义: 对同一类型的多个对象传递同一个消息 (调用同名方法), 却能得到不同响应的现象称为多态性; 方法的重载和方法的重写及动态绑定是多态的实现方式;
- 2) 向上转型: 声明一个类型的引用对象名以后, 可以创建该类型以及该类型的子类型的对象赋值给它;
- 3) 动态绑定: 通过对象名调用成员方法时, 无论声明时对象的类型是什么, 调用的方法一定是创建对象 (new 操作) 时对应类的方法; 如果在子类中重写了父类中的成员方法, 在向上转型的情景中, 使用父类对象名调用的成员方法都是子类中定义的方法。

3、抽象类和抽象方法

- 1) abstract 关键字用于定义抽象类和抽象方法;
- 2) 定义抽象类: [访问权限] abstract class 类名 { 类体 }
- 3) 抽象方法定义格式: [访问权限] abstract 方法类型 方法名(参数列表); 没有方法体!
- 4) 抽象类中可以定义抽象方法, 也可以定义非抽象方法; 抽象方法只能定义在抽象类或接口中;
- 5) 抽象类的子类如果不是抽象类, 需要实现父类中全部的抽象方法, 如果子类还是一个抽象类, 可以实现父类中部分抽象方法;
- 6) 抽象类不能被实例化, 即不能创建抽象类的对象;
- 7) 抽象类只有被继承才有意义, 结合向上转型和动态绑定实现多态。

4、接口

- 1) **interface** 关键字用于定义接口：[访问权限] **interface** 接口名 { 接口体 }
- 2) 接口中的所有方法都默认具有 **public abstract** 特性
- 3) 接口中的所有变量都默认具有 **public static final** 特性
- 4) 接口的实现格式及要求：接口只有被实现才有意义
 - 一个类在定义时通过 **implements** 关键字可以实现一个或多个接口
 - 实现接口的非抽象类需要对接口中的所有方法给出完整的定义
 - 可以声明接口类型的对象，但不能创建接口对象；结合向上转型和动态绑定实现多态。

五、Java 的异常处理机制和 IO 流

- 1、Java 中异常类的组织结构：java.lang.Throwable->Exception, Error
- 2、try~catch~finally 结构的执行过程
- 3、throw 语句：主动抛出异常
- 4、Java 允许自定义异常类，自定义的异常类须是 Exception 的子类
- 5、Java 中的 IO 流：字节流、字符流
 - 1) 字节流：OutputStream InputStream
 - 2) 字符流：Reader Writer